

LAB REPORT 10

More About Functions in C++

Computer Programming · UET Nowshera · Electrical Engineering

1 Objective

This lab explored more advanced function techniques in C++. The key topics covered were:

- Understanding and applying **Function Call by Value**
- Understanding and applying **Function Call by Reference**
- Using **default parameter values** in function declarations

2 Theoretical Background

Call by Value

In **call by value**, a copy of each argument is passed into the function. The function works entirely on these copies, so any modifications inside the function have no effect on the original variables in the calling code. This is the default behaviour in C++ and is appropriate whenever a function only needs to read or use a value without modifying the caller's data.

Call by Reference

In **call by reference**, the function receives direct access to the memory address of the original variable rather than a copy. Any changes made inside the function immediately affect the original variable. This is indicated by an ampersand (&) after the parameter type. Call by reference is also a practical way to let a single function return more than one result, since multiple reference parameters can all be modified at once.

Default Parameter Values

A function declaration can assign a default value to its trailing parameters. When the function is called without supplying that argument, the default is used automatically. If the caller does provide a value, the default is ignored. Default parameters reduce the need for multiple overloaded versions of the same function and make calls more concise for common use cases.

3 Software Used

IDE

DEV-C++ IDE

COMPILER

C++ Compiler (bundled with DEV-C++)

4 Experimental Tasks**Task 1 — Function with Parameters**

A void function named multiply accepts two integer parameters. The product is calculated inside the function and printed. Called from main() with arguments 7 and 8.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
void multiply(int num1, int num2) {
    cout << "Product of " << num1 << " and " << num2
        << " is: " << num1 * num2 << endl;
}
int main() {
    multiply(7, 8);
    return 0;
}
```

Task 2 — Function with Return Value

A function named `getSquare` takes an integer and returns its square. In `main()`, the function is called with 9, the result is stored, and then printed.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int getSquare(int number) {
    return number * number;
}
int main() {
    int val = 9;
    int sq = getSquare(val);
    cout << "Square of " << val << " is: " << sq << endl;
    return 0;
}
```

5 Conclusion

This lab deepened the understanding of functions by examining call by value, call by reference, and default parameter values. Call by reference proved especially useful for modifying caller variables directly or effectively returning multiple results in one call. Together these techniques make functions more flexible and expressive, forming an important part of clean, well-structured C++ programs.